

## 24BCF10059 - Lakshdeep

### Experiment 4

#### Aim :

To study and implement advanced SQL techniques such as conditional/custom ordering using CASE expressions inside ORDER BY, and self-joins for modelling hierarchical (manager–employee) relationships.

#### Question 1

You are given a table tbl\_happiness that contains the happiness ranking of different countries.

- (i) Retrieve all the records from the table while ensuring that India and Sri Lanka appear at the top, followed by the rest of the countries in their original ranking order.
- (ii) Make all the rankings in descending order, except India and Sri Lanka.

#### Solution :

(i)  
SELECT rankings , country  
FROM tbl\_happiness  
ORDER BY  
CASE WHEN country IN( 'India' , 'Sri Lanka')  
THEN 0 ELSE 1 END,  
Rankings;

(ii)  
SELECT rankings , country  
FROM tbl\_happiness  
ORDER BY  
CASE WHEN country IN ( 'India ' , 'Sri Lanka ' )  
THEN 0 ELSE 1 END ,  
CASE WHEN country IN ( 'India ' , 'Sri Lanka ' )  
THEN rankings END ASC ,  
CASE WHEN country NOT IN ( 'India ' , 'Sri Lanka ' )  
THEN rankings END DESC

## Question 2

A company maintains employee information in a single table where each employee may report to another employee acting as their manager. Prepare a report that displays the details of every employee who supervises at least one other employee. The report should include the manager's employee ID, name, total number of direct reports, and the average age of those reporting employees. The average age should be rounded to the nearest whole number, and the final result should be sorted by employee ID.

### Solution :

```
SELECT E.employee_id , E.name
FROM Employees AS E
WHERE (
    SELECT COUNT(reports_to)
    FROM Employees AS E1
    WHERE E1.reports_to = E.employee_id
) >= 1
```

### Learning Outcomes :

After completing this experiment, the following learning outcomes were achieved.

- Understood how to implement custom/conditional sorting using CASE expressions inside ORDER BY.
- Learned to prioritise specific rows while preserving the natural order of the remaining rows.
- Implemented multi-level ORDER BY logic combining ascending and descending directions conditionally.
- Applied self-joins to model manager–employee (hierarchical) relationships within a single table.
- Used GROUP BY with COUNT() and ROUND(AVG()) to build a manager-wise summary report.
- Strengthened practical understanding of PostgreSQL query design for real-world HR and ranking datasets.